

25MT1306E - MATHEMATICS FOR DATA SCIENCE AND ANALYTICS

Project Report on

Credit Card Fraud Detection and Statistical Analysis

Using Mathematical and Statistical Methods

Under the Guidance of

DR. NAGESH

Department of Mathematics

Team Members

Name	Roll Number
Arabati Charan	2510030005
Indra Karan Reddy	2510030148
Nanda Kishor Reddy	2510030296
Thurpu Yeshwanth Reddy	2510080001

KONERU LAKSHMAIAH EDUCATION FOUNDATION

Department of Mathematics

Green Fields, Vaddeswaram, Guntur District - 522 502

June - 2026

I. Declaration by Students

We hereby declare that the project entitled "Credit Card Fraud Detection and Statistical Analysis Using Mathematical and Statistical Methods" submitted to the Department of Mathematics, KONERU LAKSHMAIAH EDUCATION FOUNDATION, is an original work carried out by us under the guidance of DR. XYZ. This project has not been submitted to any other university or institution for the award of any degree or diploma.

Name: Arabati Charan Roll No.: 2510030005 Signature: _____	Name: Indra Karan Reddy Roll No.: 2510030148 Signature: _____
Name: Nanda Kishor Reddy Roll No.: 2510030296 Signature: _____	Name: Thurpu Yeshwanth Reddy Roll No.: 2510080001 Signature: _____

II. Certificate

This is to certify that the project entitled "Credit Card Fraud Detection and Statistical Analysis Using Mathematical and Statistical Methods" is a bonafide work carried out by the following students of KONERU LAKSHMAIAH EDUCATION FOUNDATION, in partial fulfillment of the requirements for the course 25MT1306E - MATHEMATICS FOR DATA SCIENCE AND ANALYTICS.

Name	Roll Number	Signature
Arabati Charan	2510030005	_____
Indra Karan Reddy	2510030148	_____
Nanda Kishor Reddy	2510030296	_____
Thurpu Yeshwanth Reddy	2510080001	_____

Guide: DR. NAGESH

Department of Mathematics

Signature: _____

Project

May 27, 2026

[160]:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import scipy.stats as stats


df = pd.read_csv(

    "creditcard.csv"

)


#####

#### M1 Data Cleansing ####

#####


# Drop completely empty columns and rows

df.dropna(how='all', axis=1, inplace=True)

df.dropna(how='all', axis=0, inplace=True)


# Remove any 'Unnamed' columns if they appear

df.drop(columns=[c for c in df.columns if 'Unnamed' in str(c)],

errors='ignore', inplace=True)


# Check and remove duplicate rows

n_before = len(df)

df.drop_duplicates(inplace=True)

n_after = len(df)

print(f'Duplicates removed: {n_before - n_after}')


# Check missing data percentage BEFORE filling

print("--- Missing Data Percentage per Column ---")
```

```

missing_pct = (df.isnull().mean() * 100).round(2) print(missing_pct)

print("-" * 40)

# Impute any remaining missing values with column median

df.fillna(df.median(numeric_only=True), inplace=True)

# Final Checks

print(f'Remaining nulls: {df.isnull().sum().sum()}') print(f'Final Shape: {df.shape}')

print(f'Time range: {df["Time"].min()} sec -> {df["Time"].max()} sec')

# Display clean descriptive statistics print("\n--- Cleaned

Data Summary ---")

print(df[["Time", "Amount", "Class"]].describe().round(2).T)

```

```

Duplicates removed: 1081

--- Missing Data Percentage per Column ---

Time 0.0 V1 0.0 V2 0.0 V3 0.0 V4 0.0
V5 0.0 V6 0.0 V7 0.0 V8 0.0 V9 0.0 V10 0.0
V11 0.0 V12 0.0 V13 0.0 V14 0.0 V15 0.0 V16 0.0
V17 0.0 V18 0.0 V19 0.0 V20 0.0 V21 0.0 V22 0.0
V23 0.0 V24 0.0 V25 0.0 V26 0.0 V27 0.0 V28 0.0

Amount 0.0 Class 0.0 dtype: float64

-----

Remaining nulls: 0
Final Shape: (283726, 31)
Time range: 0 sec -> 172792 sec

--- Cleaned Data Summary ---

count mean std min 25% 50% 75% max
Time 283726.0 94811.08 47481.05 0.0 54201.5 84692.5 139298.0 172792.0
Amount 283726.0 88.47 250.40 0.0 5.6 22.0 77.51 25691.16
Class 283726.0 0.00 0.04 0.0 0.0 0.0 0.00 1.00

```

[161]:

```

#####

#### M2 EXPLORATORY DATA ANALYSIS & VISUALIZATIONS ####

```

```
#####

# Target key columns for EDA

cols = ["V1", "V2", "V3", "V4", "Amount", "Class"]

-----

# 1. Summary Statistics Output

-----

print("--- Clean Summary Statistics ---")

print(df[cols].describe().round(2).T)

print("\nGenerating charts on screen...")

# -----

# 2. Variable Distributions & Skewness Check Matrix

# -----

fig, axes = plt.subplots(2, 3, figsize=(14, 8))

axes = axes.flatten()

for ax, col in zip(axes, cols):

    ax.hist(df[col].dropna(), bins=40, color='steelblue', edgecolor='white', alpha=0.7)

    ax.set_title(f'Distribution of {col}', fontsize=11)

    ax.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()

plt.show()

# -----

# 3. Class Imbalance: Fraud vs Legitimate Transactions

# -----

fig, ax = plt.subplots(figsize=(8, 4))

class_counts = df['Class'].value_counts()

bars = ax.bar(['Legitimate (0)', 'Fraud (1)'], class_counts.values,

color=['steelblue', 'crimson'], edgecolor='white', width=0.5)
```

```

for bar, count in zip(bars, class_counts.values):

    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 800,

    f'{count:}, }\n({count/len(df)*100:.2f}%)', ha='center', fontsize=11)

ax.set_title('Class Distribution: Legitimate vs Fraudulent Transactions', fontsize=14)

ax.set_ylabel('Number of Transactions', fontsize=12)

ax.grid(True, linestyle='--', alpha=0.5, axis='y')

plt.tight_layout(); plt.show()

# -----

# 4. Transaction Amount by Class (Box Plot Comparison)

# -----

fig, ax = plt.subplots(figsize=(8, 5))

legit_amounts = df[df['Class'] == 0]['Amount']

fraud_amounts = df[df['Class'] == 1]['Amount']

ax.boxplot([legit_amounts, fraud_amounts], labels=['Legitimate', 'Fraud'],

patch_artist=True,

boxprops=dict(facecolor='steelblue', alpha=0.6),

medianprops=dict(color='red', linewidth=2))

ax.set_title('Transaction Amount Distribution: Legitimate vs Fraud', fontsize=14)

ax.set_ylabel('Transaction Amount (EUR)', fontsize=12)

ax.set_yscale('log')

ax.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout(); plt.show()

# -----

# 5. Annotated Correlation Heatmap Matrix

# -----

heatmap_cols = ['V1', 'V2', 'V3', 'V4', 'V14', 'V17', 'Amount', 'Class']

corr = df[heatmap_cols].corr().round(2)

fig, ax = plt.subplots(figsize=(9, 7))

```

```

im = ax.imshow(corr, cmap='coolwarm', vmin=-1, vmax=1)

fig.colorbar(im, ax=ax)

ax.set_xticks(range(len(heatmap_cols)))

ax.set_yticks(range(len(heatmap_cols)))

ax.set_xticklabels(heatmap_cols, rotation=45, ha='right')

ax.set_yticklabels(heatmap_cols)

for i in range(len(heatmap_cols)):

    for j in range(len(heatmap_cols)):

        text_color = 'white' if abs(corr.iloc[i, j]) > 0.5 else 'black'

        ax.text(j, i, f'{corr.iloc[i,j]:.2f}', ha='center', va='center',

            color=text_color, fontsize=9)

ax.set_title('Correlation Matrix (Key Features & Class)', fontsize=14)

plt.tight_layout(); plt.show()

# -----

# 6. Hourly Transaction Amount Trends (Dual Axis)

# -----

df['Hour'] = (df['Time'] // 3600).astype(int)

hourly_legit = df[df['Class']==0].groupby('Hour')['Amount'].mean()

hourly_fraud = df[df['Class']==1].groupby('Hour')['Amount'].mean()

fig, ax1 = plt.subplots(figsize=(12, 6))

line1, = ax1.plot(hourly_legit.index, hourly_legit.values,

    color='navy', linewidth=2, label='Avg Legit Amount (EUR)')

ax1.set_xlabel('Hour Since Recording Began', fontsize=12)

ax1.set_ylabel('Avg Legit Transaction Amount (EUR)', color='navy', fontsize=12)

ax1.tick_params(axis='y', labelcolor='navy')

ax1.grid(True, linestyle='--', alpha=0.4)

ax2 = ax1.twinx()

line2, = ax2.plot(hourly_fraud.index, hourly_fraud.values,

```

```

color='crimson', linewidth=2, alpha=0.8, label='Avg Fraud Amount (EUR)') ax2.set_ylabel('Avg Fraud Transaction Amount (EUR)',
color='crimson', fontsize=12) ax2.tick_params(axis='y', labelcolor='crimson')

ax2.grid(False)

lines = [line1, line2]

labels = [l.get_label() for l in lines]

ax1.legend(lines, labels, loc='upper right', frameon=True, fontsize=10) plt.title('Hourly Average Transaction Amount:
Legitimate vs Fraud', fontsize=14) plt.tight_layout(); plt.show()

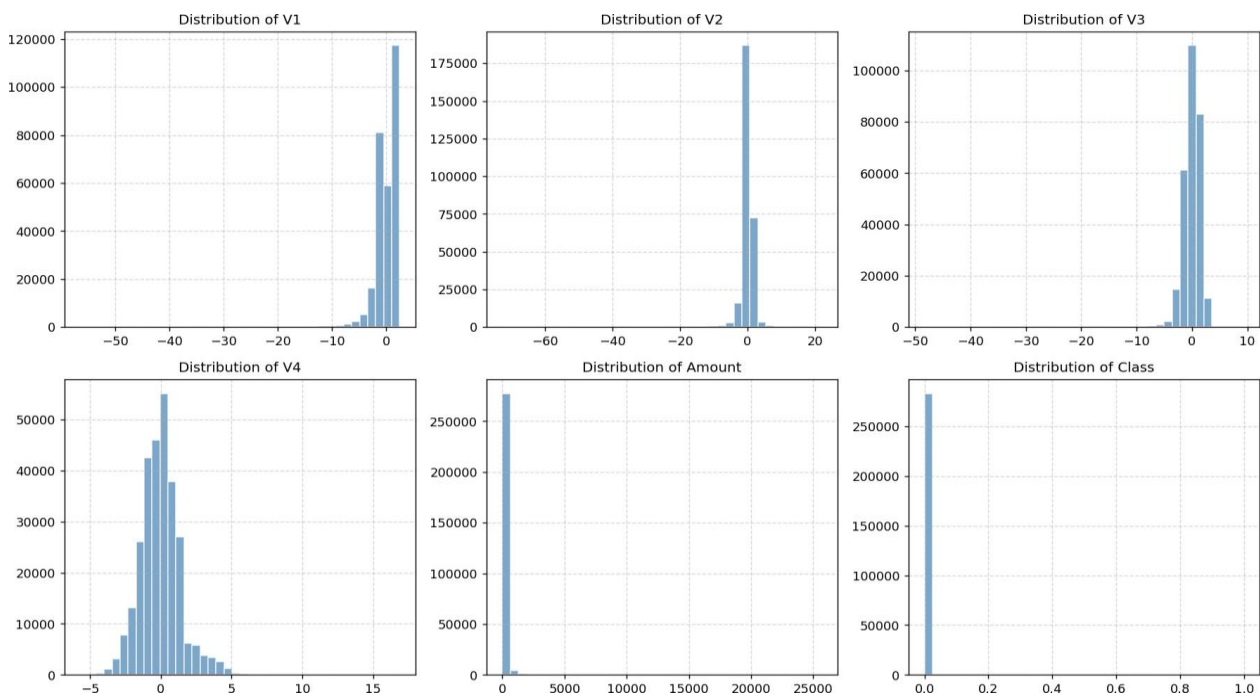
```

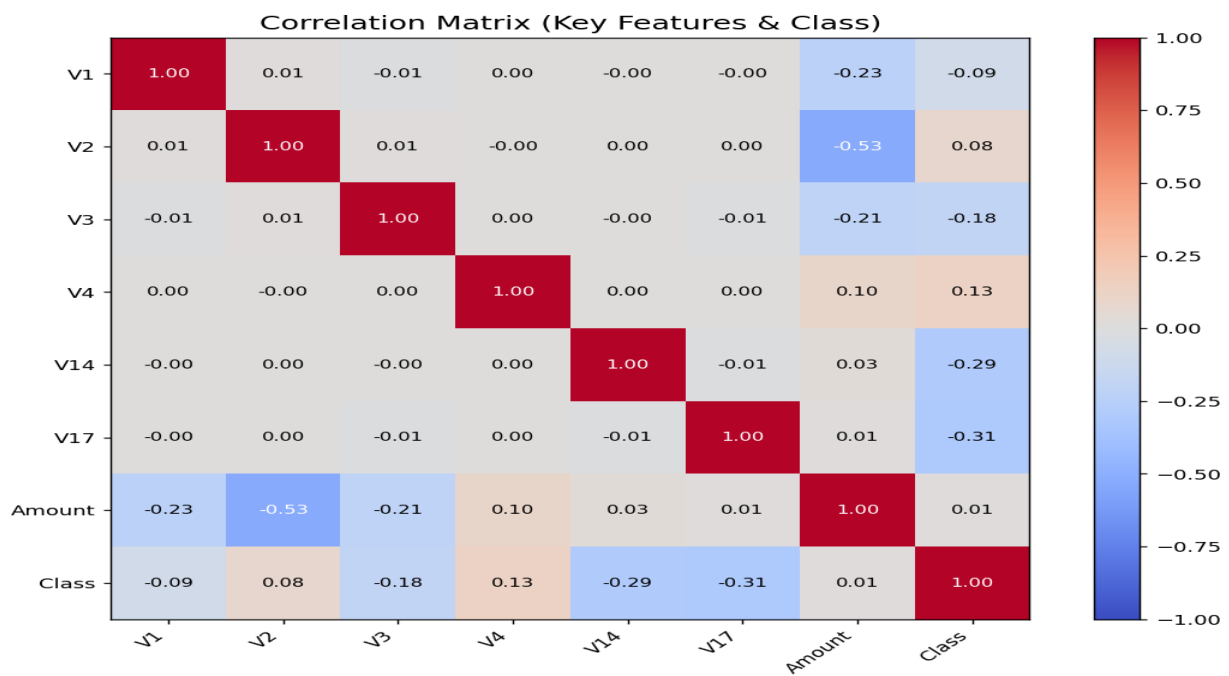
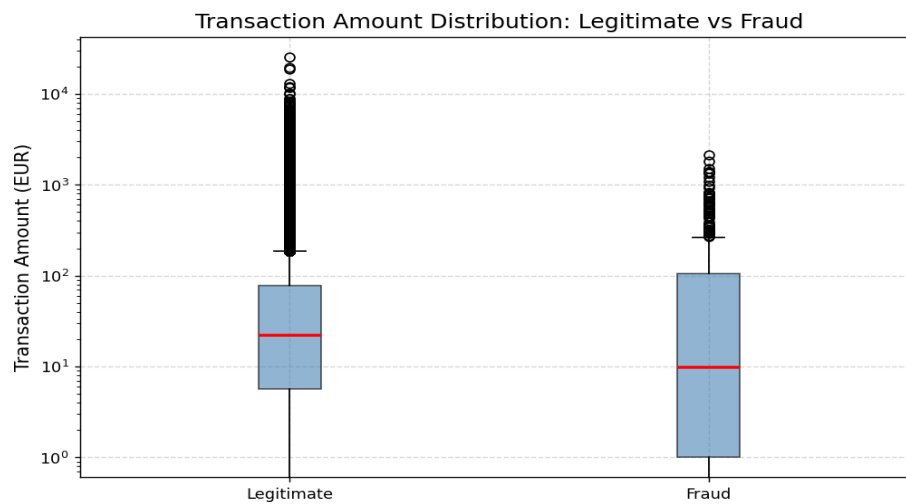
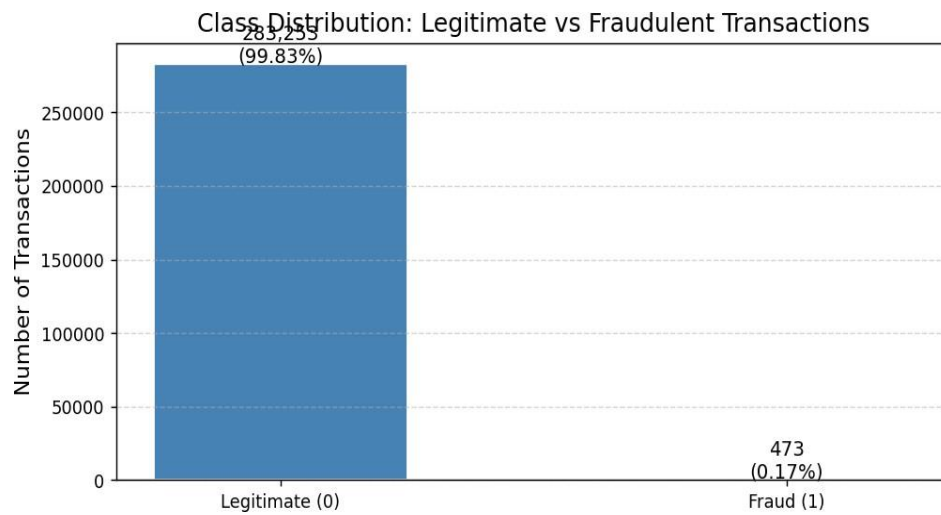
```

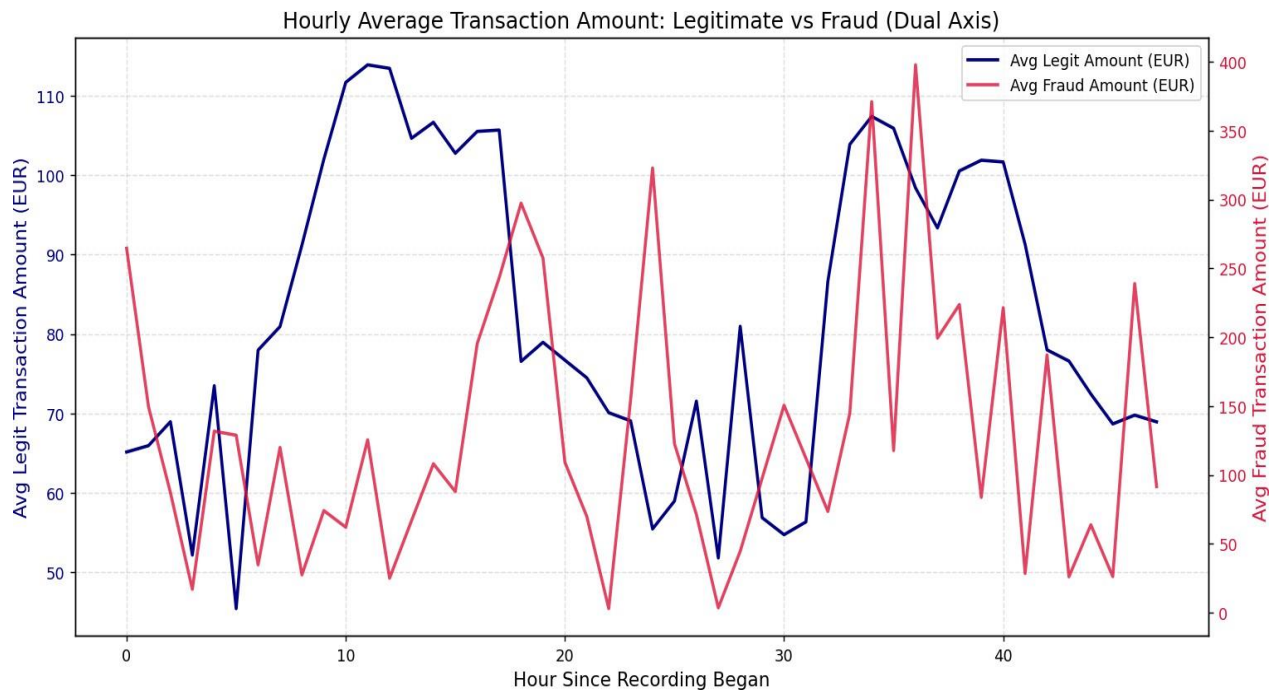
--- Clean Summary Statistics ---
count mean std min 25% 50% 75% max
V1 283726.0 0.01 1.95 -56.41 -0.92 0.02 1.32 2.45
V2 283726.0 -0.00 1.65 -72.72 -0.60 0.06 0.80 22.06
V3 283726.0 0.00 1.51 -48.33 -0.89 0.18 1.03 9.38
V4 283726.0 -0.00 1.41 -5.68 -0.85 -0.02 0.74 16.88
Amount 283726.0 88.47 250.40 0.00 5.60 22.00 77.51 25691.16
Class 283726.0 0.00 0.04 0.00 0.00 0.00 0.00 1.00

```

Generating charts on screen...







[162]:

```
#####

####M3####

#####

x = df[df['Amount'] > 0]['Amount'].dropna()

# 1. Distribution Plot plt.figure(figsize=(10,4))

plt.hist(x, bins=60, edgecolor='black', color='skyblue', alpha=0.8)

plt.axvline(x.mean(), color='red', linestyle='--', linewidth=2, label=f'Mean:
{x.mean():.2f}')

plt.axvline(x.median(), color='green', linestyle='-', linewidth=2, label=f'Median:
{x.median():.2f}')

plt.title('Distribution of Transaction Amount', fontsize=13) plt.xlabel('Amount (EUR)')

plt.ylabel('Frequency') plt.legend()

plt.grid(True, linestyle='--', alpha=0.5) plt.tight_layout()

plt.show()
```

2. Descriptive Statistics

```
print(" Descriptive Statistics (Transaction Amount) ") print(f' Sample Size : {len(x)}')
```

```
print(f' Mean : {x.mean():.3f} EUR')
```

```
print(f' Median : {x.median():.3f} EUR')
```

```
print(f' Std Dev : {x.std():.3f}')
```

```
print(f' Skewness : {x.skew():.3f}')
```

```
print(f' Kurtosis : {x.kurt():.3f}') print()
```

3. Exceedance Probabilities

```
print(" Exceedance Probabilities (Amount vs Benchmarks) ") thresholds = [100, 500, 1000,  
5000]
```

```
for t in thresholds:
```

```
    prob = (x > t).mean()
```

```
    print(f' P(Amount > {t} EUR) = {prob:.4f} ({prob*100:.2f}%)')
```

```
Descriptive Statistics (Transaction Amount)
```

```
Sample Size : 281918
```

```
Mean : 89.040 EUR
```

```
Median : 22.500 EUR
```

```
Std Dev : 251.101
```

```
Skewness : 16.938
```

```
Kurtosis : 840.268
```

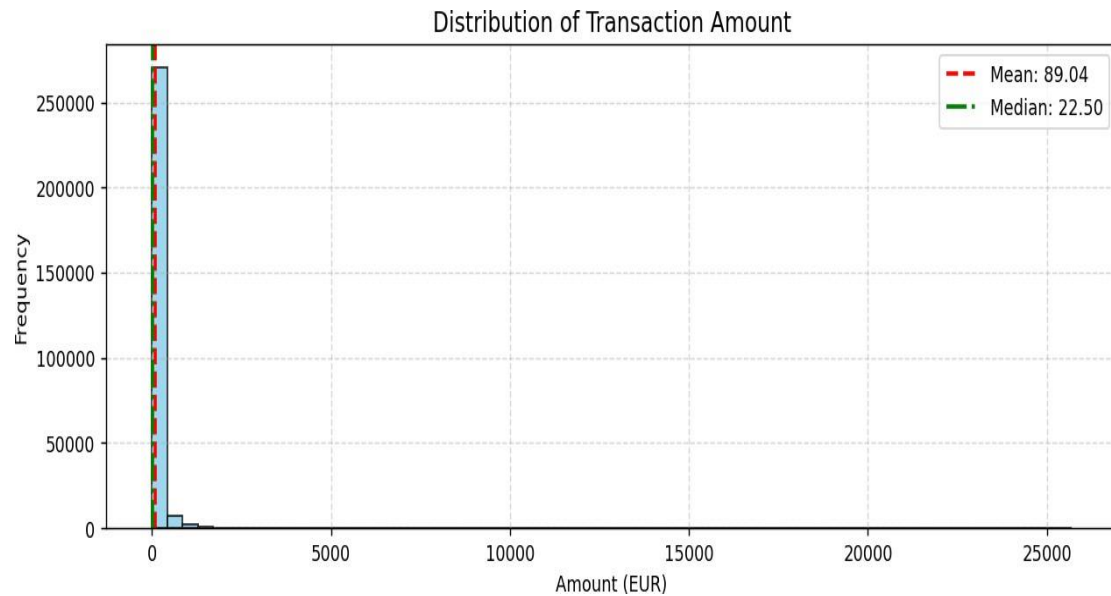
```
Exceedance Probabilities (Amount vs Benchmarks)
```

```
P(Amount > 100 EUR) = 0.2001 (20.01%)
```

```
P(Amount > 500 EUR) = 0.0323 (3.23%)
```

```
P(Amount > 1000 EUR) = 0.0104 (1.04%)
```

```
P(Amount > 5000 EUR) = 0.0002 (0.02%)
```



[163] :

```
#####

#### M4 Inference #### #####

inf_data = df.dropna(subset=['Amount', 'V14'])

# Calculate the Pearson correlation coefficient using pandas (.corr) # V14 has one of the strongest
correlations with fraud/Class

correlation = inf_data['Amount'].corr(inf_data['V14'], method='pearson') print(" M4: Correlation Inference
")

print(f'Pearson Correlation between Amount and V14: {correlation:.4f}')

# Interpret the strength of the correlation if abs(correlation) >
0.7:

strength = "Strong Positive" if correlation > 0 else "Strong Negative" elif abs(correlation) > 0.4:

strength = "Moderate Positive" if correlation > 0 else "Moderate Negative" else:

strength = "Weak"

print(f'Conclusion: There is a {strength} correlation.')
```

```

print('This suggests transaction amount alone is not a strong predictor of fraud.')

# Bonus: Correlation between V17 and Class

corr_v17 = df['V17'].corr(df['Class'], method='pearson')

print(f'\nPearson Correlation between V17 and Class (Fraud): {corr_v17:.4f}') if abs(corr_v17) > 0.3:

print('V17 shows a meaningful linear relationship with fraud classification.')

```

M4: Correlation Inference

Pearson Correlation between Amount and V14: 0.0341

Conclusion: There is a Weak correlation.

This suggests transaction amount alone is not a strong predictor of fraud.

Pearson Correlation between V17 and Class (Fraud): -0.3135

V17 shows a meaningful linear relationship with fraud classification.

[164]:

```

#####

#### M5 Regression #### #####

# Filter valid rows

reg_data = pd.DataFrame({'Amount': df['Amount'], 'Time': df['Time']}) reg_data =

reg_data[(reg_data['Amount'] > 0)].dropna()

X = reg_data['Time']

y = reg_data['Amount']

# Simple linear regression with a single scipy call

slope, intercept, r_value, p_value, std_err = stats.linregress(X, y) r_squared = r_value ** 2

print(" Linear Regression: Predicting Transaction Amount based on Time ") print(f'Equation : Amount =

{slope:.6f} * Time + {intercept:.4f}') print(f'R-squared : {r_squared:.4f}')

print(f'p-value : {p_value:.4e}')

```

```

if p_value < 0.05:

print('Result: Statistically significant relationship (p < 0.05).') else:

print('Result: No statistically significant linear relationship.')

```

```

Linear Regression: Predicting Transaction Amount based on Time
Equation : Amount = -0.000058 * Time + 94.4961
R-squared : 0.0001
p-value : 7.7041e-09
Result: Statistically significant relationship (p < 0.05).

```

[165]:

```

#####

#### M6 Visualization ####

#####

# Create a clean figure grid with 2 subplots fig, axes = plt.subplots(1, 2,

figsize=(16, 5))

# Plot 1: Linear Regression Line (Time vs Amount)

sample = reg_data.sample(n=5000, random_state=42) # Sample for visual clarity axes[0].scatter(sample['Time'],

sample['Amount'], alpha=0.1, color='gray', label='Data Points')

x_vals = np.linspace(X.min(), X.max(), 100) y_vals = slope * x_vals

+ intercept

axes[0].plot(x_vals, y_vals, color='red', linewidth=2, label=f'Fit:

y={slope:.5f}x+{intercept:.2f}') axes[0].set_title('Effect of Time on Transaction Amount')

axes[0].set_xlabel('Time (seconds)') axes[0].set_ylabel('Transaction Amount (EUR)')

axes[0].legend()

axes[0].grid(True, linestyle='--', alpha=0.5)

# Plot 2: Hourly Average Transaction Amount Profile

```

```

df_clean = df[df['Amount'] > 0].copy()

df_clean['Hour'] = (df_clean['Time'] // 3600).astype(int) hourly_profile =

df_clean.groupby('Hour')['Amount'].mean() axes[1].plot(hourly_profile.index,

hourly_profile.values, marker='o', color='teal', linewidth=2) axes[1].set_title('Average

Transaction Amount by Hour') axes[1].set_xlabel('Hour Since Recording Began')

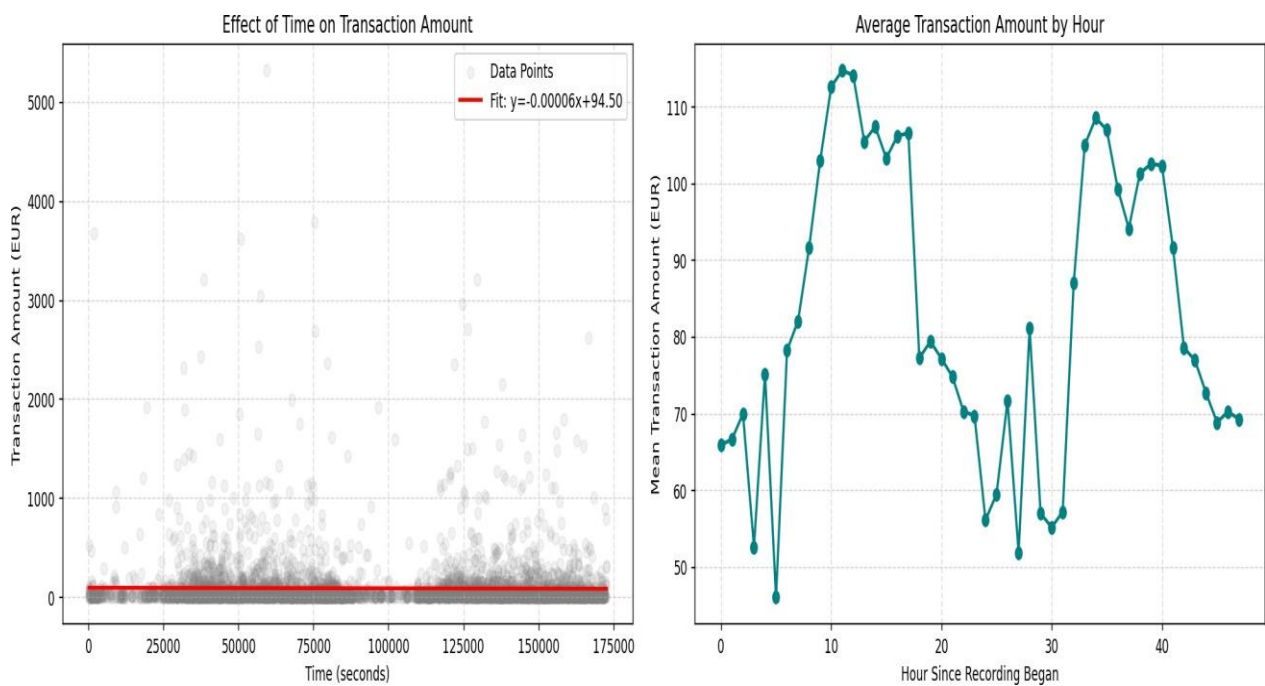
axes[1].set_ylabel('Mean Transaction Amount (EUR)') axes[1].grid(True, linestyle='--',

alpha=0.5)

plt.tight_layout()

plt.show()

```



Updated Project Report Addendum

This section updates the previous project review document using the newly uploaded Python file and the uploaded creditcard.csv dataset. The original M1 to M6 flow is retained, and the changes from the revised code are added here with updated results and IEEE references.

A. Updated Dataset and Cleaning Results

Item	Updated value
Original dataset shape	284,807 rows x 31 columns
Duplicates removed	1,081
Final cleaned shape	283,726 rows x 31 columns
Remaining null values	0
Time range	0 sec to 172792 sec
Legitimate transactions	283,253 (99.83%)
Fraud transactions	473 (0.17%)

B. Changes Added in the Updated Python Code

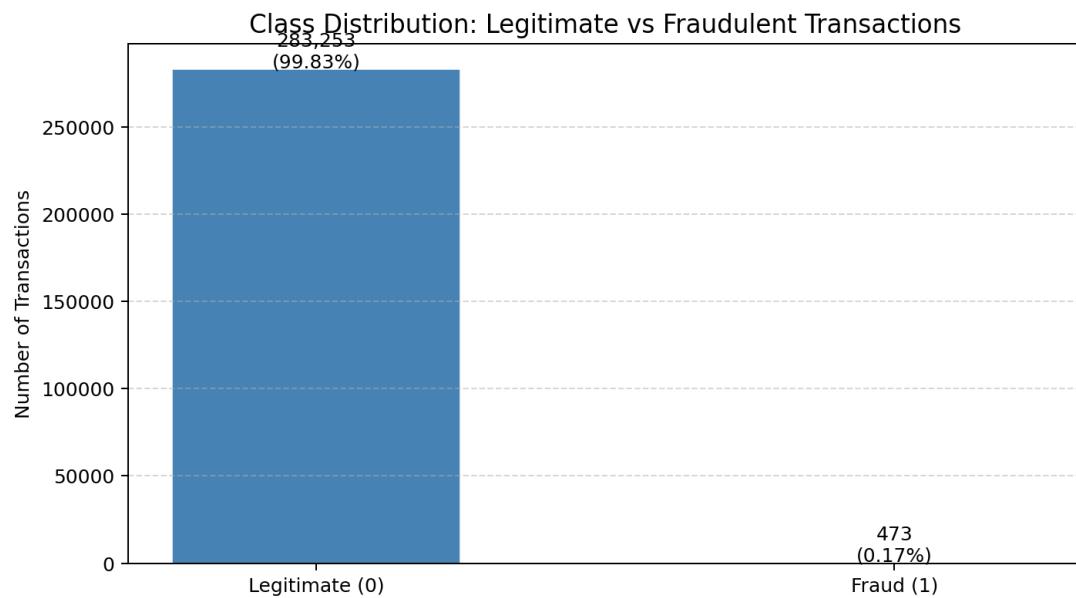
- Updated chart labels and axis labels to use euro notation for transaction amount.
- Retained the M1-M6 structure but improved comments and section naming for readability.
- Updated the fraud interpretation sentence to state that transaction amount alone is not a strong predictor of fraudulent behaviour.
- Added clearer dual-axis hourly average amount visualization for legitimate and fraud transactions.
- Kept regression and hourly profile visualization as the final M6 visual summary.

C. Updated Statistical Results

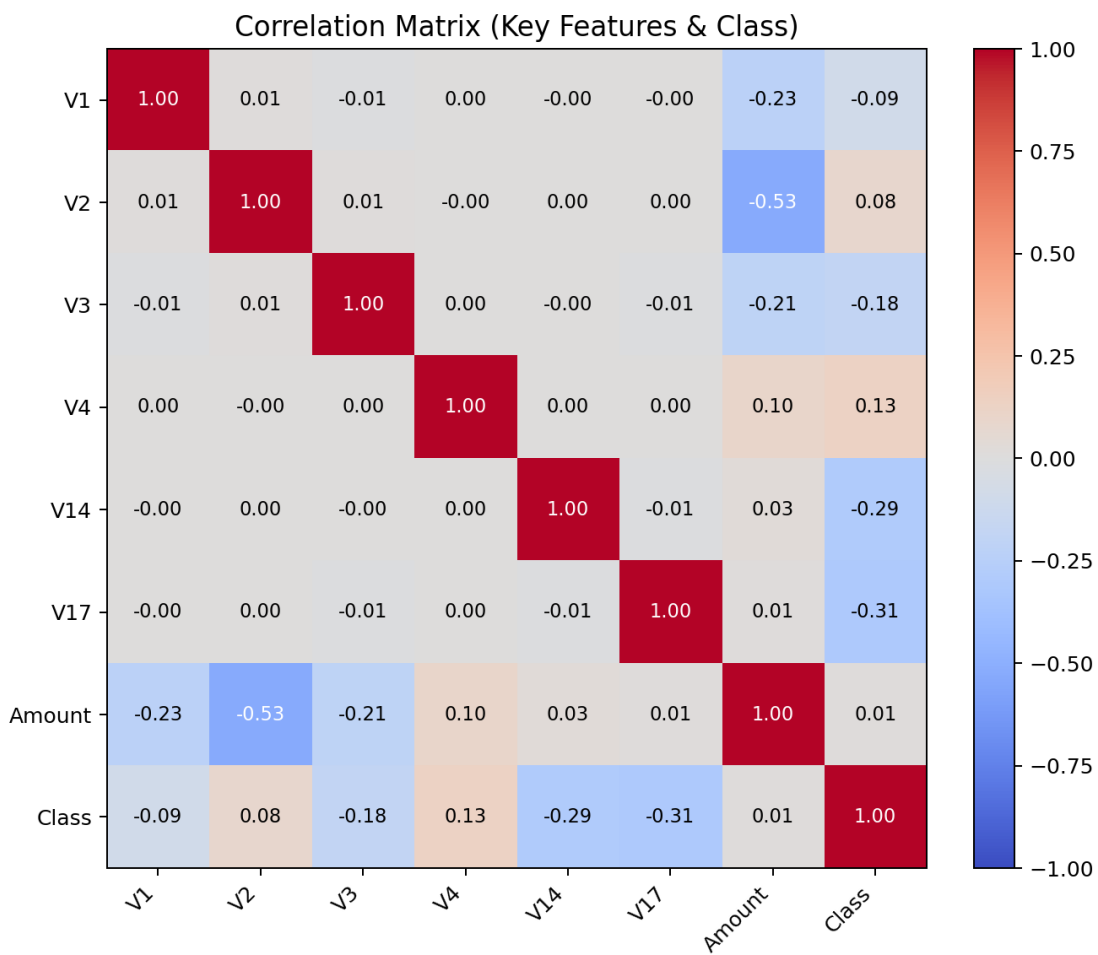
Measure	Value
Amount sample size > 0	281,918
Mean amount	89.040 EUR
Median amount	22.500 EUR
Standard deviation	251.101
Skewness	16.938
Kurtosis	840.268
Pearson Amount vs V14	0.0341
Pearson V17 vs Class	-0.3135
Regression equation	Amount = -0.000058 * Time + 94.4961
R-squared	0.0001
p-value	7.7041e-09
Benchmark	Exceedance probability
Amount > 100 EUR	20.01%
Amount > 500 EUR	3.23%
Amount > 1000 EUR	1.04%
Amount > 5000 EUR	0.02%

Interpretation: Amount is highly right-skewed, and the regression p-value is statistically significant mainly because the dataset is very large. The R-squared value is almost zero, so Time has very low practical predictive power for Amount.

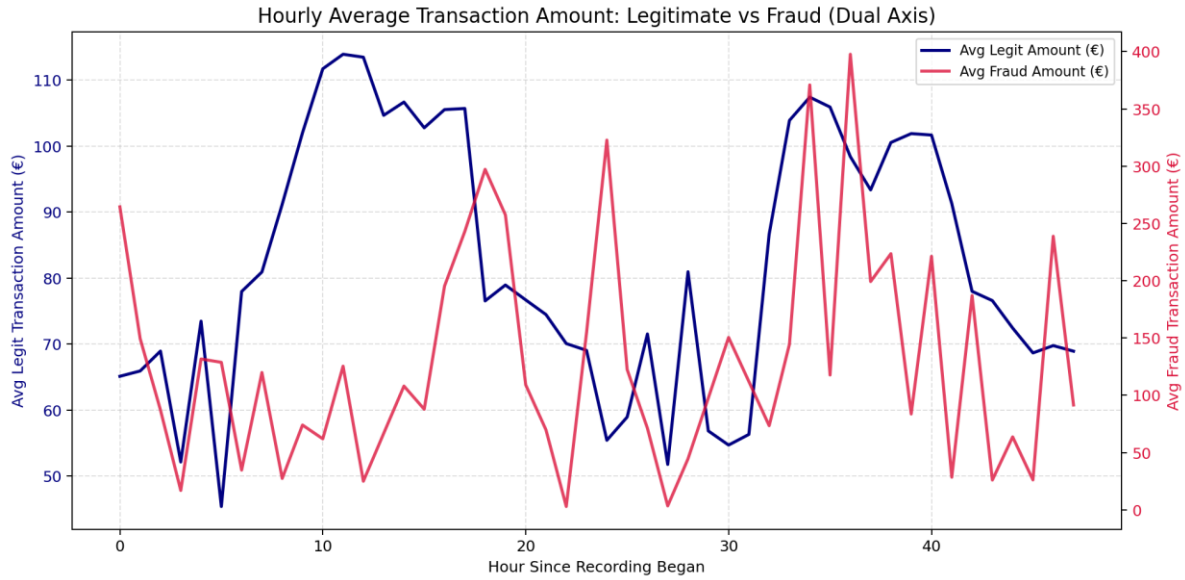
D. Updated Visual Outputs



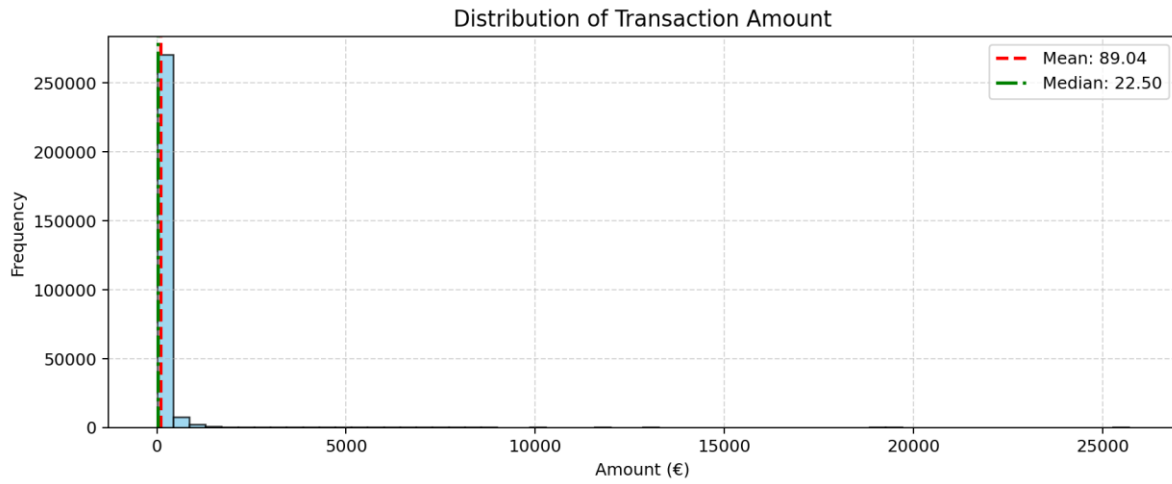
Updated Figure 1. Class distribution after duplicate removal.



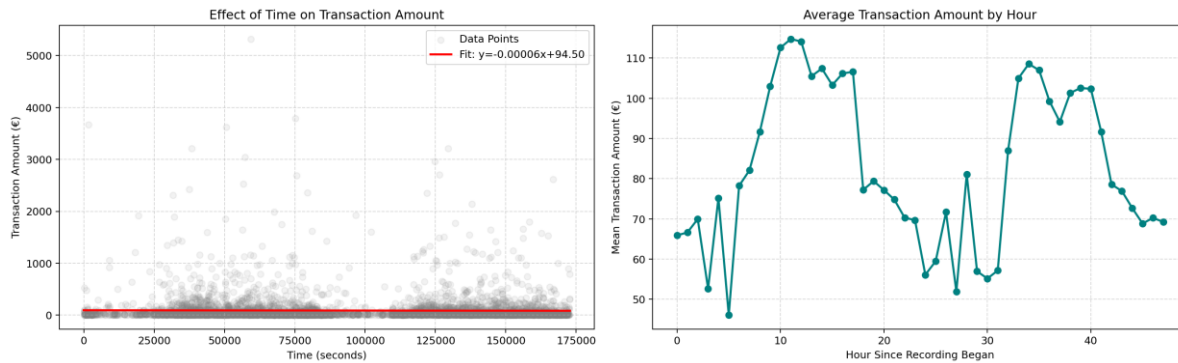
Updated Figure 2. Correlation matrix for key features and Class.



Updated Figure 3. Dual-axis hourly average amount trend.



Updated Figure 4. Transaction amount distribution with mean and median.



Updated Figure 5. Regression line and hourly amount profile.

E. Updated Conclusion

The updated analysis confirms that the dataset is clean after duplicate removal and has no remaining null values. Fraud transactions remain extremely rare at about 0.17%, making class imbalance the main analytical challenge. Transaction Amount is highly skewed, and Amount alone is not a strong fraud predictor. V17 shows a more meaningful relationship with fraud class than Amount. The Time-Amount regression is statistically significant, but its R-squared value is almost zero, so it has weak practical value.

Citation note: The dataset description and class imbalance context are based on the Kaggle/MLG-ULB dataset source [1]. The implementation uses pandas, NumPy, SciPy, and Matplotlib, which are cited as [2]-[5].

Background literature on credit card fraud detection and imbalance is cited as [6].

F. IEEE References

[1] Machine Learning Group - ULB, "Credit Card Fraud Detection," Kaggle. [Online]. Available:

<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>. Accessed: Jun. 24, 2026.

[2] W. McKinney, "Data Structures for Statistical Computing in Python," Proc. 9th Python in Science Conf., pp.

56-61, 2010.

[3] S. van der Walt, S. C. Colbert, and G. Varoquaux, "The NumPy array: a structure for efficient numerical computation," *Computing in Science & Engineering*, vol. 13, no. 2, pp. 22-30, 2011.

[4] P. Virtanen et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261-272, 2020.

[5] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90-95, 2007.

[6] X. Niu, L. Wang, and X. Yang, "A comparison study of credit card fraud detection: supervised versus unsupervised," *arXiv:1904.10604*, 2019.